# Lightning Network

## Formal verification of a payment protocol

Léo Louistisserand
Joint work with Simon Jeanteur and Matteo Maffei

LORIA (this work was done at TU Wien)

15/04/2024



# Context



Bitcoin's lack of scalability → off-chain protocols



Protocol not proven → attacks



Our goal : prove the security of a fix

# Core idea



Lock coins on the chain



Exchange this money off-chain



Use the chain to cash in

# Opening a channel



| Funding transaction | |
|---|---|
| Input | Output |
| Alice: 5<br>Bob: 5 | A&B: 10 |



| Closing transaction | |
|---|---|
| Input | Output |
| A&B: 10 | Alice: 5<br>Bob: 5 |

# Updating a channel



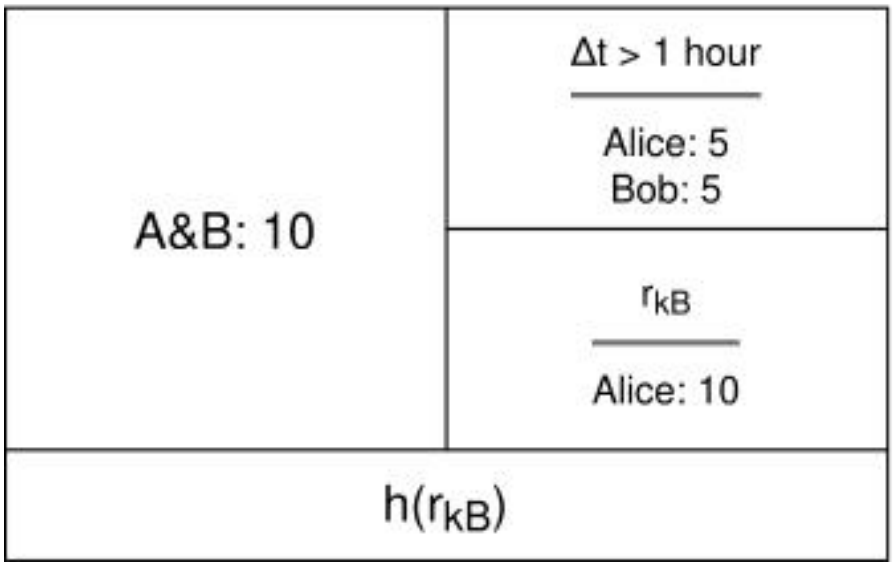
| Old transaction | |
|---|---|
| A&B: 10 | Alice: 5<br>Bob: 5 |



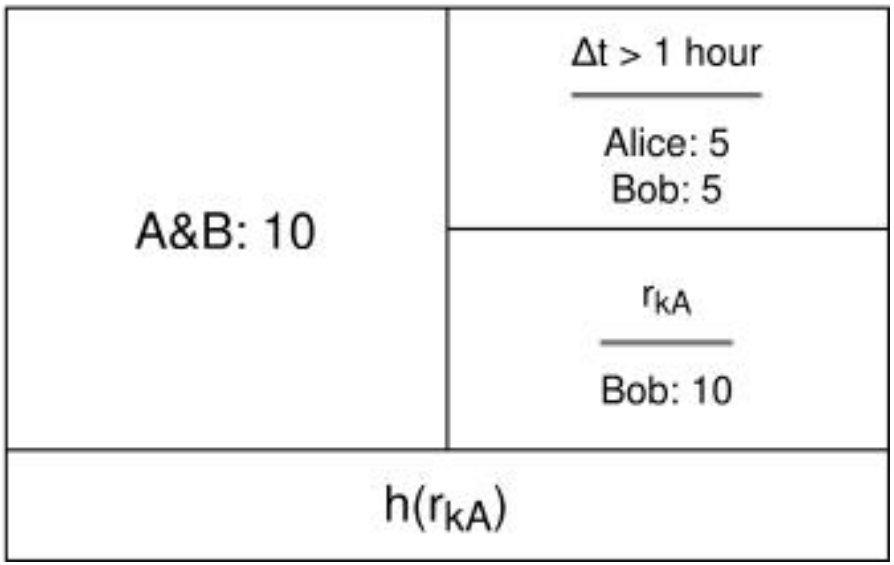
| New transaction | |
|---|---|
| A&B: 10 | Alice: 4<br>Bob: 6 |

# Revocation mechanism

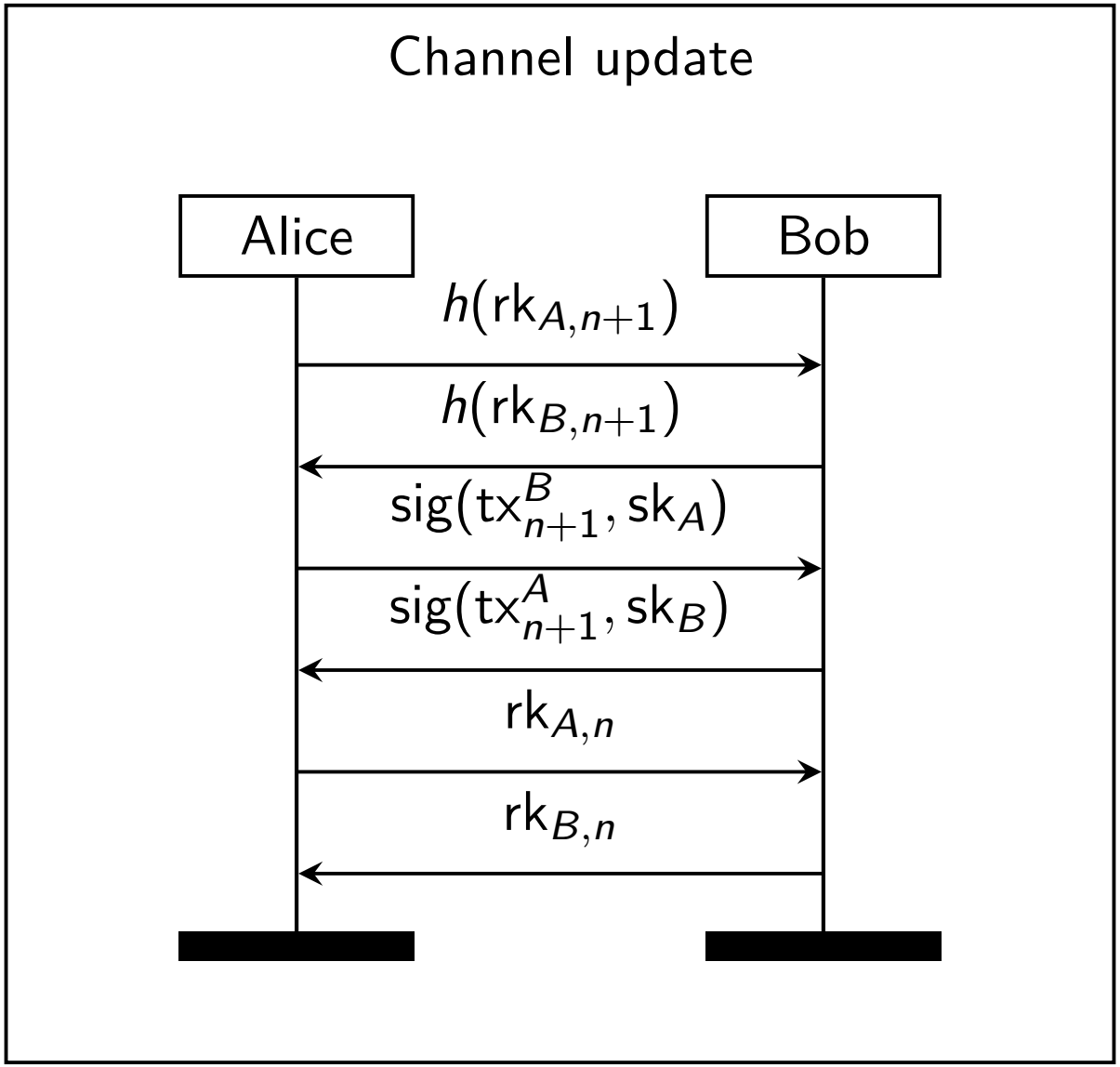
Signed by Alice, held by Bob



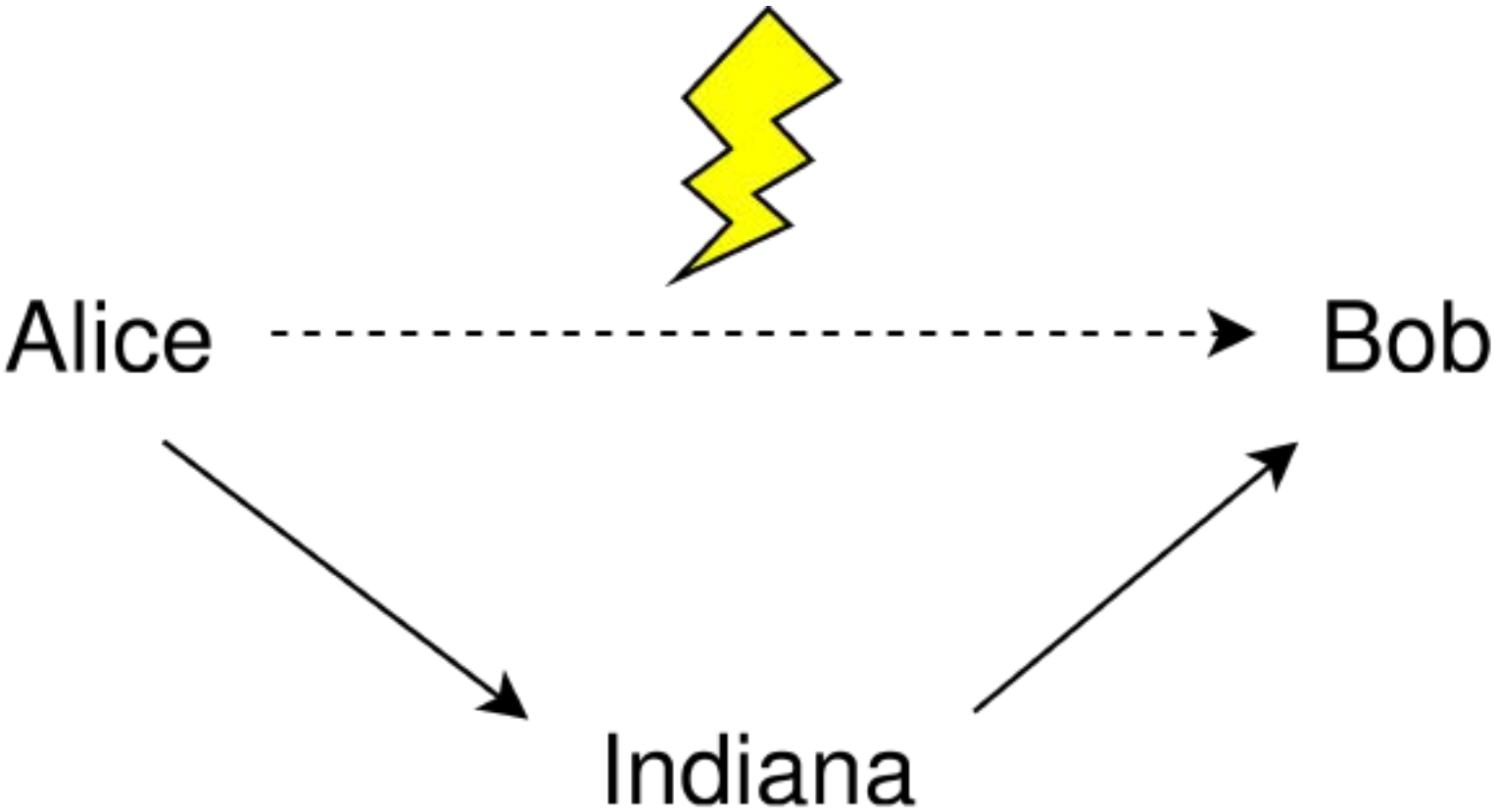

Signed by Bob, held by Alice



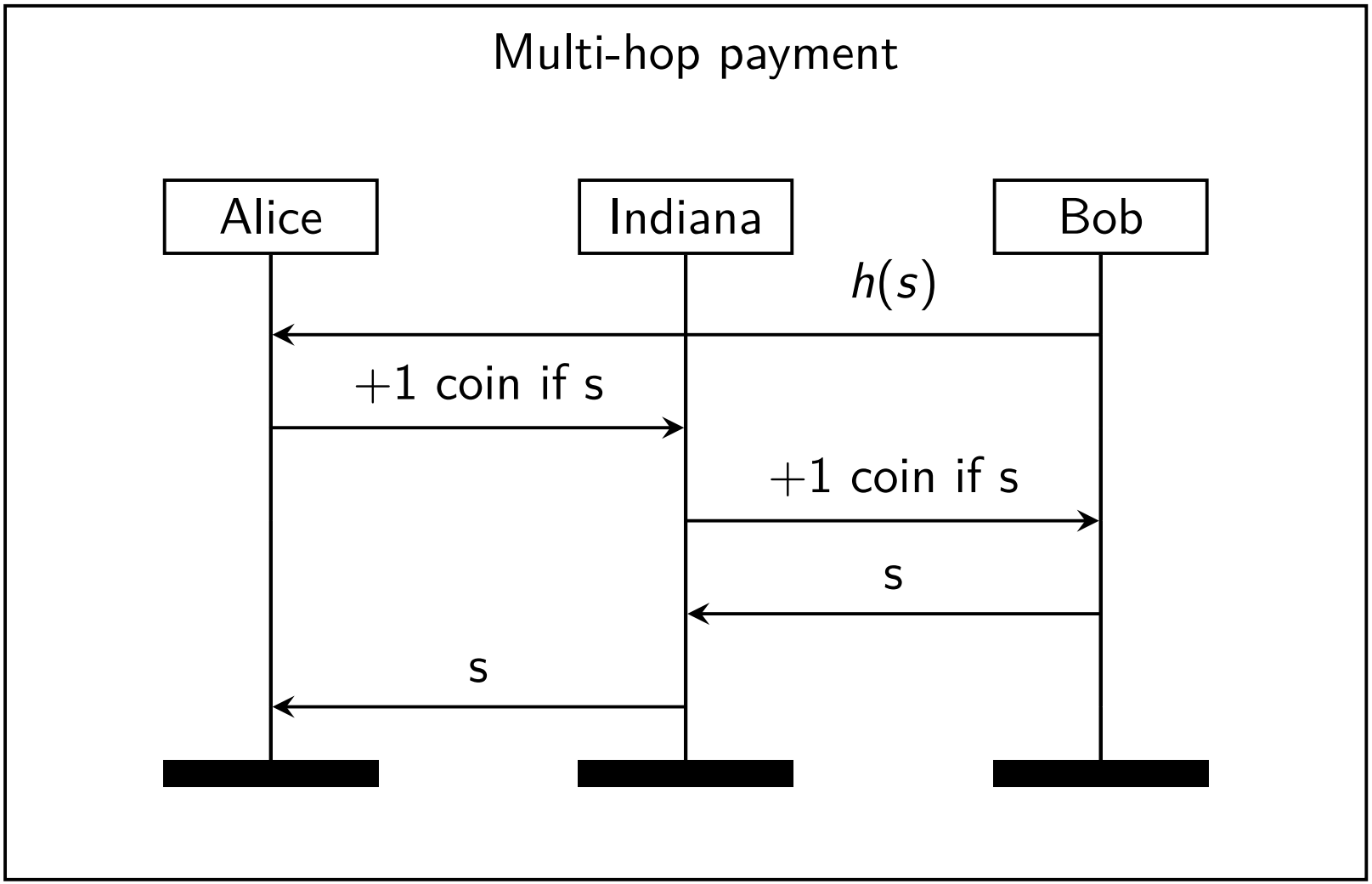
# Revocation mechanism



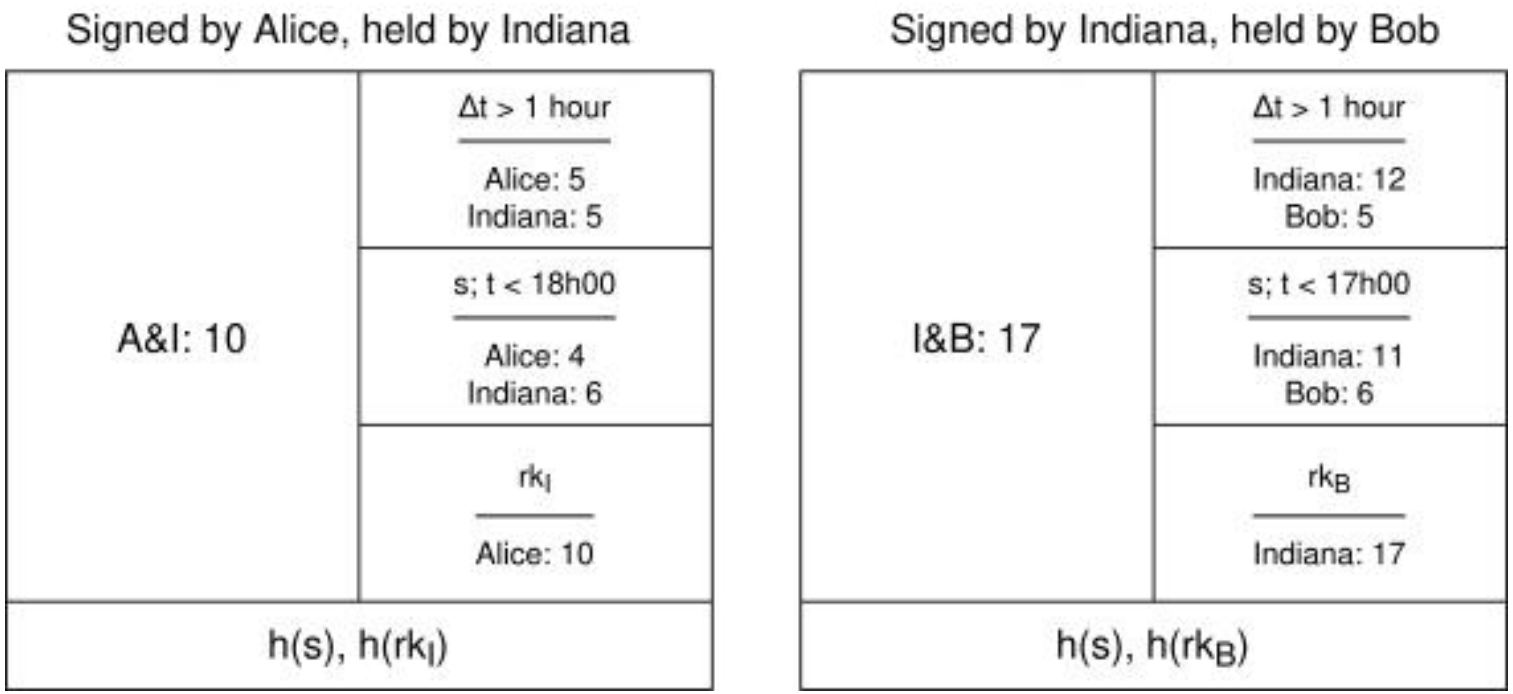
# Lightning Network



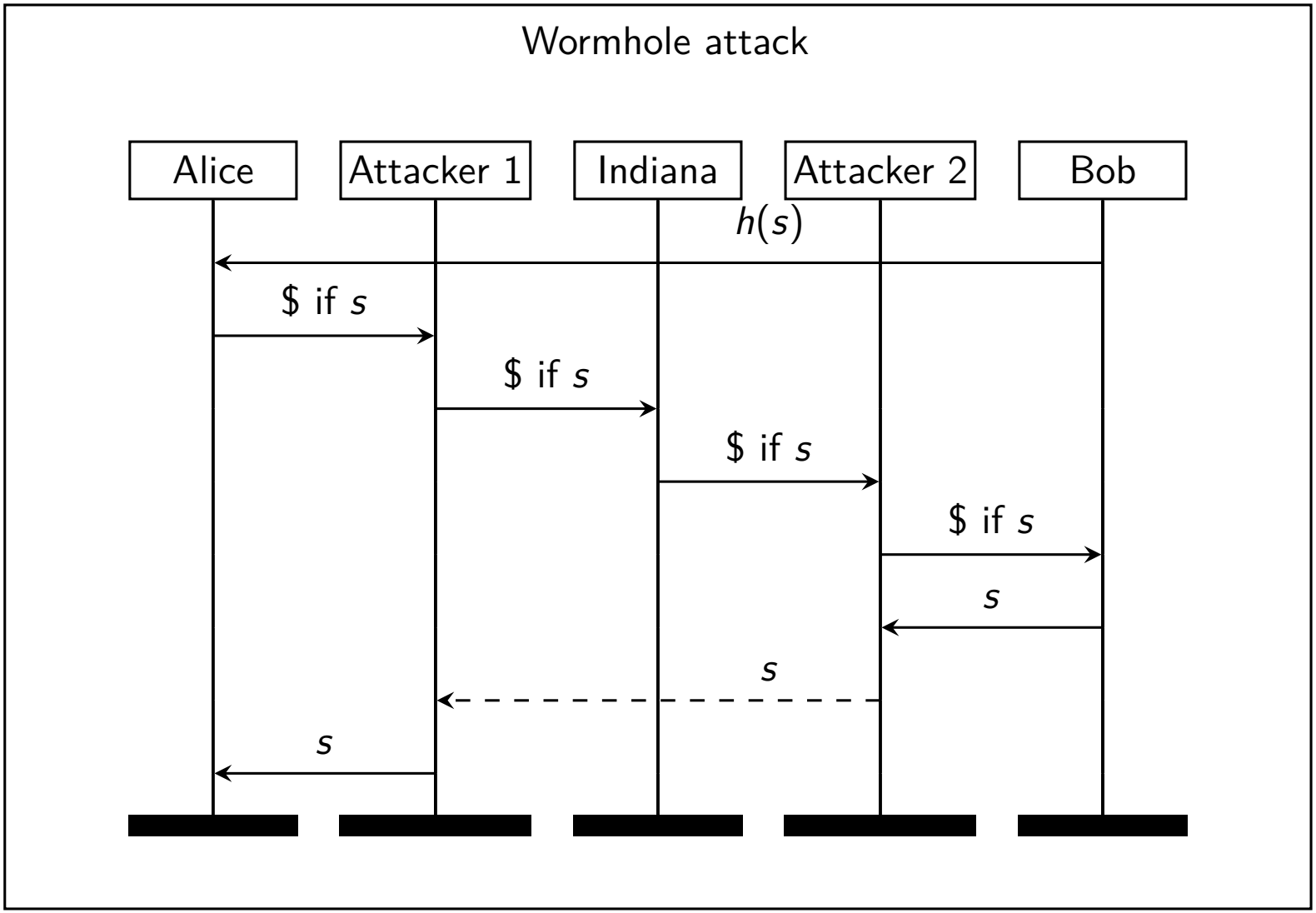
# Multi-hop payment



# Hashed Timelock Contract



# Wormhole attack



# Desired properties

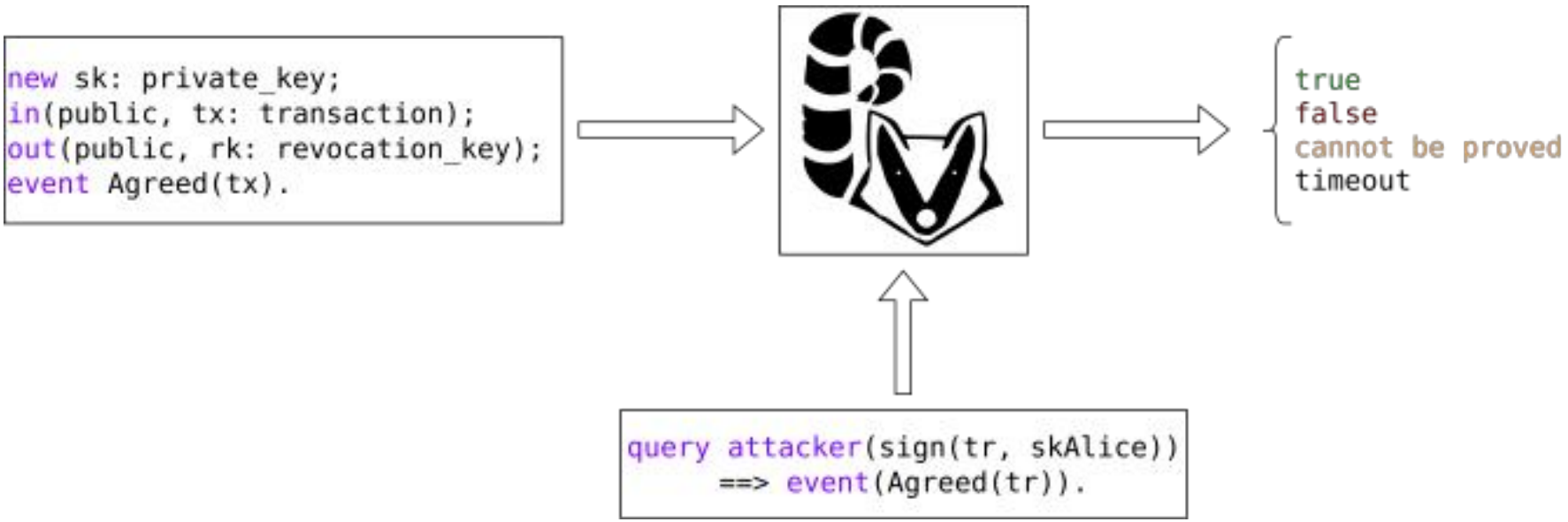


Honest participants cannot lose money

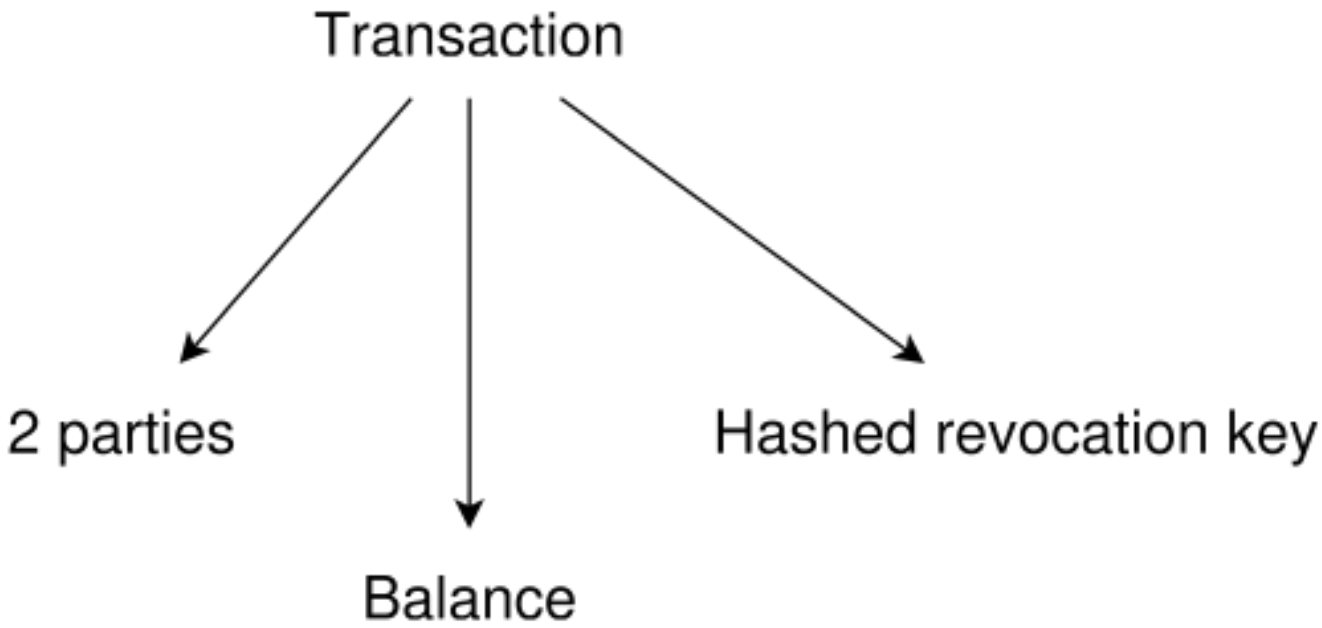


Honest participants get their fees

# The Proverif tool

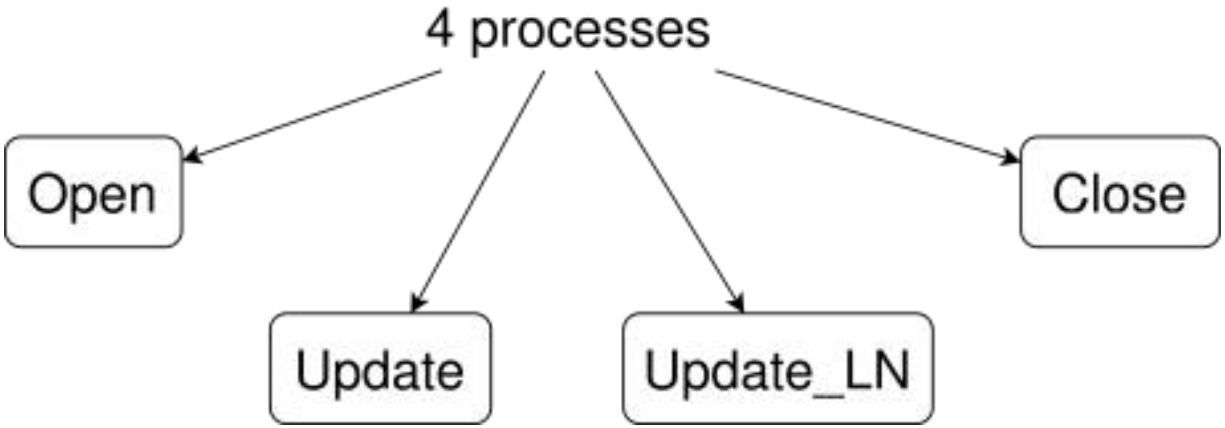


# Modelling a transaction




Transaction is represented by a quadruplet $\rightarrow$ tx = (pk1, pk2, b, h)

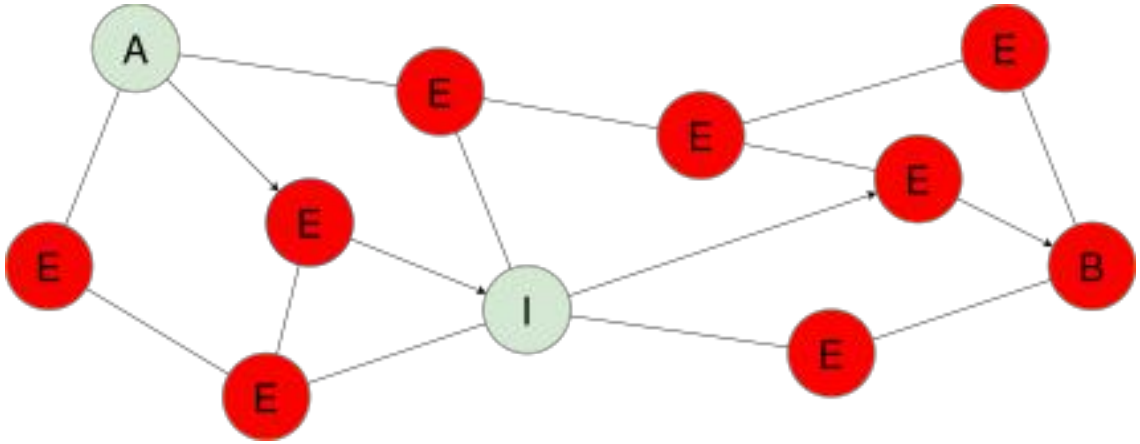
# Modelling a payment channel




- The current state is stored: revocation key, current balance, other party's signature.
- Old states are stored through events.

```
event Alice_knows(rk).
```

# Modelling the whole network




Threat model:

- Honest agents communicate via authenticated and secret channels
- All agents can be compromised

# Modelling the properties

- Indiana cannot lose money
  - **No money blocked**: Indiana can always close the channel
  - **No punishment possible**: Attacker cannot punish Indiana
  - **Defense against old states**: Indiana can punish old transactions
  - **Unforgeability**: attacker cannot forge transaction
- Indiana gets the fee
  - **Atomicity**: when Alice has paid, Indiana is able to debit

# Difficulties

| | |
|---|---|
| Time | Not modeling it |
| Passing the state | Using events |
| Liveness property | Tweat it into a correspondance property |
| Unbounded number of agents | Reduction to a bounded model |

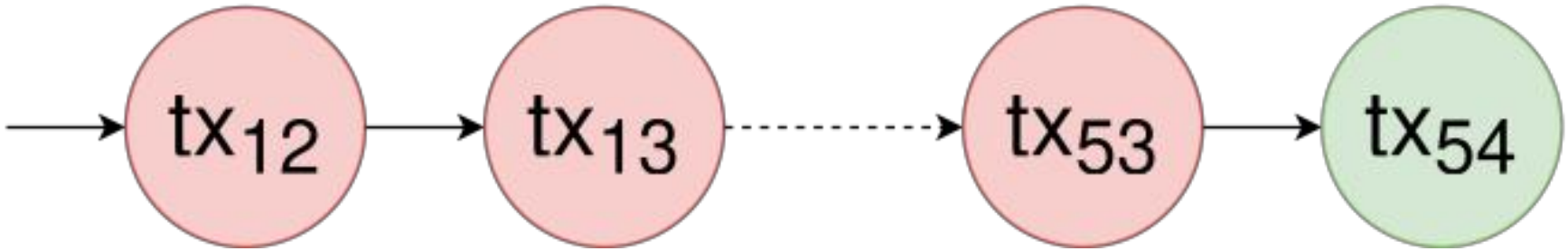
# Trick



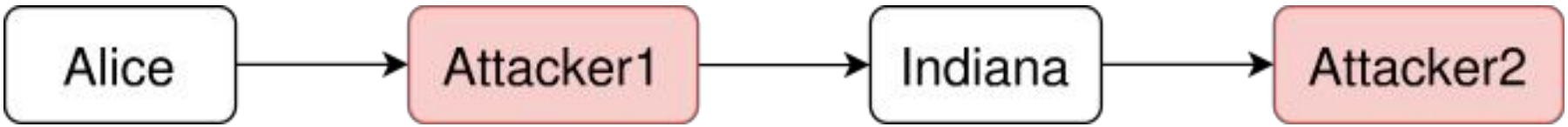
Liveness property: Indiana always holds a non-revocated transaction



Correspondance property: if transaction n Indiana holds is revocated, Indiana holds transaction n+1



# Reduced model




**Theorem**

Attack on the full network $\Longrightarrow$ attack on a 4-agent chain

The attacker can simulate processes thanks to oracles.

```
let signing_oracle(sk: private_key) =
    in(public, tx: transaction);
    event oracle_signs(tx, sk);
    out(public, sign(tx, sk)).
```

# Results

| Property | Result | Time |
|---|---|---|
| No money blocked | ✓ | <5s |
| No punishment | ✓ | <5s |
| Defense against old states | ✓ | <5s |
| Unforgeability | ✓ | <5s |
| Atomicity | ✓ | <5s |

# Conclusion



Modeling the LN protocol



Expressing all properties as correspondance property



Using a reduced model and a pen-and-paper proof



Next step: take time into account